

1-private n -Party AND from 5 random bits

Samuel Dittmer* and Rafail Ostrovsky†

Abstract

In the field of information-theoretic cryptography, randomness complexity is a key metric for protocols for private computation, that is, the number of random bits needed to realize the protocol. Although some general bounds are known, even for the relatively simple example of n -party AND, the exact complexity is unknown.

We improve the upper bound from Couteau and Rosén in Asiacrypt 2022 on the (asymptotic) randomness complexity of n -party AND from 6 to 5 bits, that is, we give a 1-private protocol for computing the AND of n parties' inputs requiring 5 bits of additional randomness, for all $n \geq 120$. Our construction, like that of Couteau and Rosén, requires a single source of randomness.

Additionally, we consider the modified setting of Goyal, Ishai, and Song (Crypto '22) where helper parties without any inputs are allowed to assist in the computation. In this setting, we show that the randomness complexity of computing a general boolean circuit C 1-privately is exactly 2 bits, and this computation can be performed with seven helper parties per gate.

1 Introduction

We are concerned with a concrete question in information-theoretic cryptography: For $n \geq 3$, how many bits of randomness are required for n parties with inputs $a_1, a_2, \dots, a_n$ to compute the bitwise AND of their inputs $a_1 \wedge a_2 \wedge \dots \wedge a_n$ in a 1-private protocol with semi-honest security? This question was first asked explicitly in [8], although a construction in [9] established that $O(1)$ bits were sufficient for any linear size circuit, thus including an AND function. Since [8] showed that at least 2 random bits are necessary, the asymptotic randomness complexity is known to be $\Theta(1)$.

One practical motivation behind this question is to study techniques for reducing randomness complexity. In real world cryptography, random bits must either be collected from physical sources or generated from a pseudo-random number generator applied to some initial seed. There are costs and security risks associated with both approaches, see e.g. [5, 6] and the discussion in [4].

Our setting is idealized in many ways. By studying information-theoretic cryptography, we remove any reliance on cryptographic hardness assumptions. We assume

*Stealth Software Technologies, Inc. Email: sam@stealthsoftwareinc.com

†UCLA, Email: rafail@cs.ucla.edu

only a single corrupted party, and semi-honest behavior from that party. And we wish only to compute the n -party AND, as opposed to some more complex circuit. It is therefore remarkable that, even in this setting, the exact randomness complexity is unknown.

There is a noteworthy connection between this question and in circuit complexity. As Kushilevitz, Ostrovsky, and Rosén showed [9], a boolean function f that requires $\omega(1)$ bits of randomness to securely compute in this setting would also require a circuit of size $\omega(1)$, see e.g. [8] for further discussion and other connections between randomness complexity and other complexity measures.

1.1 Notation and preliminaries

Note that treating the bits as elements over $\mathbb{F}_2$, for bits a, b we can write $a \oplus b = a + b$, $a \wedge b = ab$, so that $a_1 \wedge a_2 \wedge \cdots \wedge a_n := a_1 a_2 \cdots a_n$.

In this work, we improve the upper bound on the randomness complexity of n -party AND for all but finitely many n . To clarify this, we write β_n for the randomness complexity of n -party AND for fixed n , and denote $\beta := \limsup_{n \rightarrow \infty} \beta_n$. We show here that $\beta_n \leq 5$ for $n \geq 120$ and thus that $\beta = 5$.

1.2 Our Contribution

Our main result is the following:

Theorem 1.1. *There exists a 1-private semi-honest protocol with 5 random bits for n parties to compute the AND of their respective inputs, for $n \geq 120$.*

Which gives as an immediate corollary

Corollary 1.2. $\beta \leq 5$.

Additionally, we show how to modify our construction when we have access to “helper parties”:

Theorem 1.3. *For any boolean functionality f on inputs from n parties, corresponding to a circuit C with $|C|$ gates, including at least one AND gate, the randomness complexity for f in the presence of helper parties is exactly 2, and there exists a 1-private semi-honest protocol with 2 random bits that securely computes f in the presence of $O(|C|)$ helper parties.*

1.3 Prior work

Information-theoretic Cryptography. For 1-private protocols, we have a $\theta(n)$ bound on the randomness complexity if and only if the boolean function f to be computed on the bits $a_1, \dots, a_n$ can be represented by a linear circuit [9]. For the case of 1-private n -party XOR, the randomness complexity is known to be equal to exactly 1. Kushilevitz and Mansour considered the case of t -private n -party XOR protocols in [7], and showed a lower bound of $\Omega(t)$ and an upper bound of $O(t^2 \log(n/t))$. Canetti,

Kushilevitz, Ostrovsky and Rosén showed that any linear size circuit can be computed t privately for any $t < n/3$ -collusion with $O(\text{poly}(t) \cdot \log n)$ random bits [2]. t -privacy for XOR was also considered by Goyal, Ishai, and Song [4], who gave a lower bound of $\Omega(t^2)$ and an explicit construction of a protocol with $O(t^2 \log^2 n)$ random bits.

n -party AND. The problem of computing upper and lower bounds for β was first introduced explicitly in [8], which showed $2 \leq \beta \leq 8$, although the construction in [9] gives an implicit proof of an upper bound of $\beta \leq 73$. This upper bound was improved by Couteau and Rosén to $\beta \leq 6$ in [3]. Couteau and Rosén also improved on [8] by reducing the number of randomness sources, that is, the number of distinct parties that may flip coins in the protocol, from 2 to 1. Our protocol likewise uses a single randomness source, which flips all coins at the start of the protocol.

The construction of [1] showed $\beta_{2n} \leq 9$ and $\beta_{2n-1} \leq 6$, for $n \geq 2$. They give a construction that is *sequential*, meaning that the sequence of messages sent is independent of the randomness and the party inputs, and *stateless*, meaning that each message is only a function of the previous message, private bit, and distributed randomness. In this sense, our construction is *sequential* but not *stateless*; each message is a polynomial expression of all previous messages the party has received, along with the private bit and initial randomness.

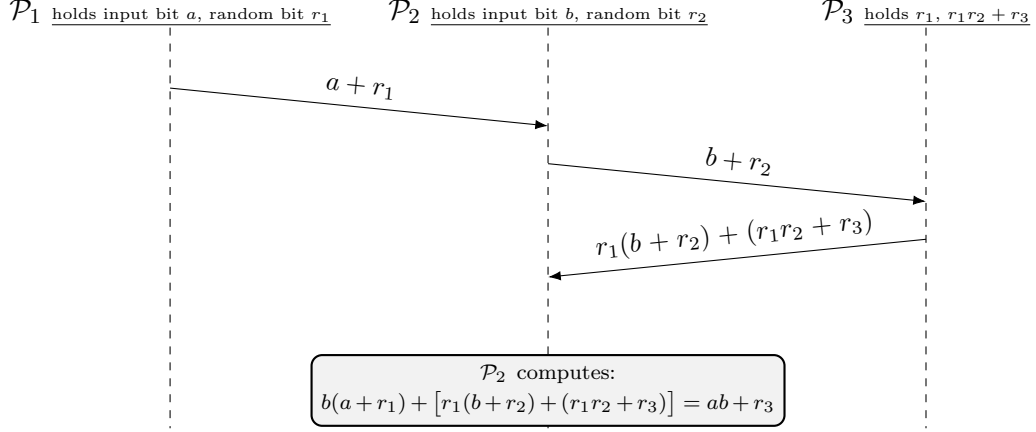
Helper parties in information-theoretic cryptography. In [4], Goyal, Ishai, and Song consider the problem of t -private computation of arbitrary boolean circuits C where in addition to the n parties holding the inputs, there are an additional collection of “helper parties” that are willing to support the computation but have no inputs of their own. They give an existence proof for a t -private protocol requiring $O(t^2 \log |C|)$ random bits and $O(t^4 |C| \log |C| \log^3 t |C|)$ helper parties, and an explicit realization of a t -private protocol requiring $O(t^2 \log^2 t |C|)$ random bits and $O(t^3 |C| \log |C| \log^2 t |C|)$ helper parties.

1.4 Overview of techniques

Our construction works on the same overall principle as [9, 8, 3]. As in these works, a sequence of small sub-committees each perform a $O(1)$ -party sub-protocol, updating from shares of $\prod_{j=1}^i a_j$ to $\prod_{j=1}^{i+1} a_j$. This is coupled with a relabeling scheme, so that randomness can be re-used from protocol to protocol while ensuring that, from the perspective of any fixed party, all of the randomness is fresh. Finally, the relabeling scheme has to be realized and aligned with committee assignments to verify that the messages sent in the update protocol are consistent across all parties’ views.

Masked Multiplication Our “masked multiplication” protocol is a secure computation between parties $\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3$, using correlated randomness generated by a dealer party $\mathcal{D}$. $\mathcal{P}_1$ begins with $a + r_1$, that is, with the secret value a masked by r_1 , and $\mathcal{P}_2$ holds the secret value b . After the protocol, $\mathcal{P}_2$ holds the masked value $ab + r_3$. Our protocol uses 3 bits of randomness; the bit r_2 is used to construct correlated randomness held by $\mathcal{P}_3$, who helps to facilitate the computation.

Protocol: Masked Multiplication $\Pi(\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3, a, b, r_1, r_2, r_3)$



We give this protocol below in Figure 2, along with a more detailed description.

First Warm-up: Helper Parties. As an immediate application of our masked multiplication protocol, we obtain a protocol for computing n -party AND with n helper parties and 3 bits of randomness. We iteratively perform the masked multiplication protocol on parties $\mathcal{P}_{i-1}, \mathcal{P}_i$, and a helper party H_{i-1} , using the random bits (r_1, r_2, r_3) when i is even and (r_3, r_2, r_1) when i is odd. We give this protocol below in Figure 1.

Modifying this construction to use only 2 bits of randomness follows the same general approach, but requires a more involved masked multiplication with six helper parties per AND gate. We show this alternate masked multiplication protocol in Section 5, and the compiler from a masked multiplication protocol to a multiparty computation protocol for an arbitrary Boolean functionality.

Second Warm-up: Rotating Randomness. To remove the helper protocols from our party, we instead assign existing parties to play an additional role as a helper party. Formally, we choose some permutation $\sigma : \{2, \dots, n\} \rightarrow \{2, \dots, n\}$ and replace the party H_{i-1} in Figure 1 with the party $\mathcal{P}_{\sigma(i)}$. Each party $\mathcal{P}_i$ will now participate in the multiplication protocol (up to) 3 times, playing the role of $\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3$. Instead of switching back and forth between (r_1, r_2, r_3) and (r_3, r_2, r_1) , we can assign a permutation $\tau(i) \in S_3$, for $i = 2, \dots, n$, and perform a masked multiplication:

$$\Pi \left(\mathcal{P}_{i-1}, \mathcal{P}_i, \mathcal{P}_{\sigma(i)}, \prod_{j=1}^{i-1} a_j, a_i, r_{\tau(i)[1]}, r_{\tau(i)[2]}, r_{\tau(i)[3]} \right)$$

To confirm that a choice (σ, τ) leads to a valid n -party AND protocol, we must verify the alignment of the committee assignments (determined by σ) and the rotating randomness (determined by τ). If we write down the views of a party $\mathcal{P}_i$ playing each of these three roles, we obtain a set of conditions on σ and τ . At this point, we begin searching for valid committee assignments and randomness through a combination of intuition, trial and error, and computer search.

Helper Party Warm-Up Protocol

Input. n parties $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_n$ and $n-1$ helper parties $H_1, \dots, H_{n-1}$ hold the following data: $\mathcal{P}_i$ holds a_i , and $\mathcal{P}_1$ holds r_1, r_2, r_3 .

Output. All parties $\mathcal{P}_i$ hold $\prod_{j=1}^n a_j$.

Protocol.

1. $\mathcal{P}_1$ flips three random coins $\mathbf{r} := (r_1, r_2, r_3)$.
2. $\mathcal{P}_1$ sends r_2 to $\mathcal{P}_i$ for all parties $\mathcal{P}_i$, with $i \leq 2 \leq n$.
3. For $i \leq 2 \leq n$ and i even, $\mathcal{P}_1$ sends r_1 and $r_1 r_2 + r_3$ to H_{i-1} .
4. For $i \leq 2 \leq n$ and i odd, $\mathcal{P}_1$ sends r_3 and $r_3 r_2 + r_1$ to H_{i-1} .
5. For $i \leq 2 \leq n$, parties $\mathcal{P}_{i-1}$, $\mathcal{P}_i$, and H_{i-1} perform a masked multiplication

$$\Pi \left(\mathcal{P}_{i-1}, \mathcal{P}_i, H_{i-1}, \prod_{j=1}^{i-1} a_j, a_i, r_1, r_2, r_3 \right) \quad \text{for } n \text{ even,}$$

and

$$\Pi \left(\mathcal{P}_{i-1}, \mathcal{P}_i, H_{i-1}, \prod_{j=1}^{i-1} a_j, a_i, r_3, r_2, r_1 \right) \quad \text{for } n \text{ odd.}$$

6. At the conclusion of this loop, $\mathcal{P}_n$ holds $\prod_{j=1}^n a_j + r_1$ if n is odd and $\prod_{j=1}^n a_j + r_3$ otherwise. Either way, $\mathcal{P}_n$ sends this value to $\mathcal{P}_1$.
7. $\mathcal{P}_1$ adds r_1 or r_3 (based on the parity of n) to the value sent from $\mathcal{P}_n$, and broadcasts $\prod_{j=1}^n a_j$ to all parties.

Figure 1: n -party AND protocol

The simplest way to define a family of permutations $\tau(i) \in S_5$ rotating the randomness used in each invocation of the masked multiplication is to apply some underlying permutation repeatedly, that is, to fix some τ and set $\tau(i) = \tau^{i-2}$ for $i = 2, \dots, n$. If the $\tau(i)$'s are on a strict rotational schedule, we will also want the assignment of helper parties $\sigma(i)$ to follow a steady pattern modulo 5. We in fact take $\sigma(i) = i + k$, for some integer k , where the addition is taken modulo $n-1$; this gives us the requirement that $n-1 \equiv 0 \pmod{5}$. We are indeed able to find such a construction, for every $n \equiv 1 \pmod{5}$.

When n is not 1 mod 5, it may appear plausible that some more complex rotational schedule for τ could be found by removing the requirement that $\tau(i) = \tau^{i-2}$. After we expended some effort searching, this began to appear less plausible to us. We turned instead to a generalization of permutations.

Main Protocol: Randomness Transformations. Instead of instructing each party $\mathcal{P}_i$ to have some view of the randomness of the protocol that amounts to a relabeling of $(r_1, \dots, r_5)$, we can treat $r_1, \dots, r_5$ as linearly independent vectors, and have $\mathcal{P}_i$'s transformed view of the r_i 's be formed by an invertible linear transformation of the r_i 's. Indeed, linear independence is the key property we need to ensure that $\mathcal{P}_i$'s view is simulatable, and acting on a linearly independent set by an invertible linear transformation will preserve this property.

We therefore replace our search for a sequence of permutations $\tau(i) \in S_5$ with a search for a sequence of linear transformations T_i satisfying the same set of conditions. We show below in our proof of Lemma 4.1 that transformations T_i can be composed while preserving the desired property. As a result, we show that we can realize any permutation σ on $\{2, \dots, n\}$ as long as its cycle decomposition is a union of 5-cycles and 31-cycles, which leads to our desired construction for $n \geq 120$.

2 Definitions

Let $\mathbf{e}_i$ be the basis vector for $\mathbb{F}_2^5$, for $1 \leq i \leq 5$. In order to capture the conditions on the pair $(\sigma, \tau[i])$ or the pair (σ, T_i) , we give the following definition:

Definition 2.1. *We say a positive integer n is 5-friendly if there exists a permutation $\sigma : \{2, \dots, n\} \rightarrow \{2, \dots, n\}$ and invertible linear transformations $T_i : \mathbb{F}_2^5 \rightarrow \mathbb{F}_2^5$, for $2 \leq i \leq n$, subject to the following four conditions:*

- **Condition 1:** $T_i^\top \mathbf{e}_3 = T_{i+1}^\top \mathbf{e}_1$ for $2 \leq i < n$,
- **Condition 2:** $T_{\sigma(i)}^\top \mathbf{e}_2 = T_i^\top \mathbf{e}_1$ for all i with $2 \leq i \leq n$,
- **Condition 3:** $T_{\sigma(i)}^\top \mathbf{e}_4 = T_i^\top \mathbf{e}_2$ for all i with $2 \leq i \leq n$,
- **Condition 4:** $T_{\sigma(i)}^\top \mathbf{e}_5 = T_i^\top \mathbf{e}_3$ for all i with $2 \leq i \leq n$.

We also say a pair (σ, T_i) is 5-friendly if it satisfies the above conditions. Recall that for any vector $\mathbf{v}$ and index k , we have $\langle T^\top \mathbf{e}_k, \mathbf{v} \rangle = \langle \mathbf{e}_k, T\mathbf{v} \rangle$ is equal to the k th coordinate of $T\mathbf{v}$.

By an aggressive abuse of notation, we define:

Definition 2.2. *For $2 \leq i \leq n$, $1 \leq j \leq 5$,*

$$T_i(\mathbf{r}, j) = \langle \mathbf{e}_j, T_i \mathbf{r} \rangle$$

The proof of our main theorem is then a consequence of the following two lemmas proven in sections 3.3 and 4.2 respectively:

Lemma 2.3. *If n is 5-friendly, $\beta_n \leq 5$.*

Lemma 2.4. *For $n \geq 120$, n is 5-friendly.*

3 The Protocol

3.1 Masked Multiplication Transition Step

Masked Multiplication $\Pi(\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3, a, b, r_1, r_2, r_3)$

Input. Three parties $\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3$ hold the following data: $\mathcal{P}_1$ holds $a + r_1$, $\mathcal{P}_2$ holds b and r_2 , and $\mathcal{P}_3$ holds r_1 and $r_1 r_2 + r_3$.

Output. $\mathcal{P}_2$ holds $ab + r_3$.

Protocol.

1. $\mathcal{P}_1$ sends $a + r_1$ to $\mathcal{P}_2$.
2. $\mathcal{P}_2$ sends $b + r_2$ to $\mathcal{P}_3$.
3. $\mathcal{P}_3$ receives $b + r_2$ and computes $r_1 \cdot (b + r_2) + (r_1 r_2 + r_3) = br_1 + r_3$, and sends the result to $\mathcal{P}_2$.
4. $\mathcal{P}_2$ receives $br_1 + r_3$ computes $b \cdot (a + r_1) + (br_1 + r_3) = ab + r_3$.

Figure 2: Transition Step: Three party masked multiplication protocol

3.2 Full Protocol

Full Protocol

Input. n parties $\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_n$ hold the following data: $\mathcal{P}_i$ holds a_i , and $\mathcal{P}_1$ holds r_1, r_2, r_3, r_4, r_5 . Additionally, all parties hold a permutation $\sigma : \{2, \dots, n\} \rightarrow \{2, \dots, n\}$ and linear transformations $T_i : \mathbb{F}_2^5 \rightarrow \mathbb{F}_2^5$ such that (σ, T_i) is 5-friendly.

Output. All parties hold $\prod_{j=1}^n a_j$.

Protocol.

1. $\mathcal{P}_1$ flips five random coins $\mathbf{r} := (r_1, r_2, r_3, r_4, r_5)$.
2. For $i \leq 2 \leq n$, $\mathcal{P}_1$ sends $T_i(\mathbf{r}, 2)$ and $T_i(\mathbf{r}, 2)T_i(\mathbf{r}, 4) + T_i(\mathbf{r}, 5)$ to $\mathcal{P}_i$.
3. For $i \leq 2 \leq n$, parties $\mathcal{P}_{i-1}$, $\mathcal{P}_i$, and $\mathcal{P}_{\sigma(i)}$ perform a masked multiplication $\Pi(\mathcal{P}_{i-1}, \mathcal{P}_i, \mathcal{P}_{\sigma(i)}, \prod_{j=1}^{i-1} a_j, a_i, T_i(\mathbf{r}, 1), T_i(\mathbf{r}, 2), T_i(\mathbf{r}, 3))$
4. At the conclusion of this loop, $\mathcal{P}_n$ holds $\prod_{j=1}^n a_j + T_i(\mathbf{r}, 3)$, and so sends this value to $\mathcal{P}_1$.
5. $\mathcal{P}_1$ adds $T_i(\mathbf{r}, 3)$ to the value sent from $\mathcal{P}_n$ and broadcasts $\prod_{j=1}^n a_j$ to all parties.

Figure 3: n -party AND protocol

3.3 Proof of Lemma 2.3

Correctness.

In Step 3 of the full protocol, party $\mathcal{P}_{\sigma(i)}$ holds $T_{\sigma(i)}(\mathbf{r}, 2) = T_i(\mathbf{r}, 1)$ and $T_{\sigma(i)}(\mathbf{r}, 2)T_{\sigma(i)}(\mathbf{r}, 4) + T_{\sigma(i)}(\mathbf{r}, 5) = T_i(\mathbf{r}, 1)T_i(\mathbf{r}, 2) + T_i(\mathbf{r}, 3)$, and party $\mathcal{P}_i$ holds a_i and $T_i(\mathbf{r}, 2)$, as required by the masked multiplication subprotocol. We prove that $\mathcal{P}_{i-1}$ holds the required inputs by induction on i . For $i = 2$, $\mathcal{P}_1$ can compute the expression $a_1 + T_2(\mathbf{r}, 1)$ locally, since $\mathcal{P}_1$ holds a_1 and $\mathbf{r}$. For the inductive hypothesis, note that at the end of the masked multiplication subprotocol, party $\mathcal{P}_i$ learns the value

$$\prod_{j=1}^i a_j + T_i(\mathbf{r}, 3) = \prod_{j=1}^i a_j + T_{i+1}(\mathbf{r}, 1),$$

so $\mathcal{P}_i$ holds the correct inputs to play the role of party $\mathcal{P}_{(i+1)-1}$ on the next step of the protocol, as desired.

Security

For $2 \leq i < n$, each party plays all three roles in the masked multiplication protocol exactly once. (Note that party $\mathcal{P}_n$ never plays the role of $\mathcal{P}_1$ in the masked multiplication protocol, which does not weaken security).

Thus, for $2 \leq i \leq n$, each party $\mathcal{P}_i$ receives the following information along with the final output bit:

1. $T_i(\mathbf{r}, 2)$,
2. $T_i(\mathbf{r}, 2)T_i(\mathbf{r}, 4) + T_i(\mathbf{r}, 5)$,
3. $\left(\prod_{j < i} a_j\right) + T_i(\mathbf{r}, 1)$,
4. $T_i(\mathbf{r}, 1)a_i + T_i(\mathbf{r}, 3)$,
5. $a_{\sigma^{-1}(i)} + T_i(\mathbf{r}, 4)$

Since the rows T_i are linearly independent, the values $T_i(\mathbf{r}, j)$ for $j = 2, 5, 1, 3, 4$ appear as uniformly random and independent random coins to $\mathcal{P}_i$, and serve to mask the five message bits $\mathcal{P}_i$ receives in the course of the protocol execution, so these messages can be simulated by independent random coin flips.

4 Constructing appropriate permutations

4.1 Warm-up construction

Before proving Lemma 2.4, we prove as a warm-up result that n is five-friendly for all $n \equiv 1 \pmod{5}$. Set

$$\sigma(i) = i + 3,$$

for all i , taking addition mod $n - 1$ and working with $2, \dots, n$ as the representatives of distinct residue classes mod $n - 1$, and construct the permutation τ as:

$$(\tau(1) || \dots || \tau(5)) = 35214$$

and define T_i to be the permutation matrix corresponding to τ^{i-2} , for $2 \leq i \leq n$.

To prove this, we first note that

$$T_{i+1}^\top \mathbf{e}_1 = \tau^{i-1}(1) = \tau^{i-2}(3) = T_i^\top \mathbf{e}_3$$

and so the first condition for 5-friendly numbers is satisfied. We then compute

$$(\tau^3(1) || \dots || \tau^3(5)) = 51423,$$

and note that this implies

$$T_{\sigma(i)}^\top \mathbf{e}_2 = T_{i+3}^\top \mathbf{e}_2 = T_i^\top \mathbf{e}_1$$

by the same argument we used to show $T_{i+1}^\top \mathbf{e}_1 = T_i^\top \mathbf{e}_3$, and the other two conditions follow similarly.

4.2 Main construction

For our general construction, we can no longer use only permutation matrices. To get further symmetry, we also require that $T_n^\top \mathbf{e}_3 = T_2^\top \mathbf{e}_1$. We then temporarily ignore the first condition $T_i^\top \mathbf{e}_3 = T_{i+1}^\top \mathbf{e}_1$ and construct σ and T_i that satisfy the other three conditions for n -friendly numbers. Our motivation for doing so is the following:

Lemma 4.1. *Suppose we have σ and T_i such that*

1. *The second, third, and fourth conditions on 5-friendly numbers are satisfied for σ and T_i ,*
2. *The number of times any given nonzero vector $\mathbf{v}^\top$ occur in the 1st or 3rd rows of any T_i are equal, that is:*

$$|\{i | T_i^\top \mathbf{e}_1 = \mathbf{v}^\top\}| = |\{i | T_i^\top \mathbf{e}_3 = \mathbf{v}^\top\}|,$$

3. *The directed multigraph on nonzero vectors in $\mathbb{F}_2^5$ given by adding a directed edge $(\mathbf{v}, \mathbf{w})$ for each i with $T_i^\top \mathbf{e}_1 = \mathbf{v}^\top$ and $T_i^\top \mathbf{e}_3 = \mathbf{w}^\top$ consists of the union of a single connected component with isolated vertices of in- and out-degree zero.*

Then there exists some permutation $\pi : \{2, \dots, n\} \rightarrow \{2, \dots, n\}$ such that σ', T' with $\sigma' := \pi^{-1} \circ \sigma \circ \pi$ and $T'_i(\mathbf{r}, j) := T_{\pi(i)}(\mathbf{r}, j)$ satisfy all conditions for 5-friendly numbers.

Proof. Regardless of our choice of π , we have e.g.

$$T'_{\sigma'(i)}(\mathbf{r}, 2) = T_{\sigma(\pi(i))}(\mathbf{r}, 2) = T_{\pi(i)}(\mathbf{r}, 1) = T'_i(\mathbf{r}, 1),$$

so the second, third, and fourth conditions of n -friendly numbers remain satisfied, and we just need to choose π such that

$$T'_{i+1}(\mathbf{r}, 1) = T_{\pi(i+1)}(\mathbf{r}, 1) = T_{\pi(i)}(\mathbf{r}, 3) = T'_i(\mathbf{r}, 3).$$

Construct a directed multigraph on nonzero vectors over $\mathbb{F}_2^5$ as in condition (3). Then $T_{\pi(i+1)}(\mathbf{r}, 1) = T_{\pi(i)}(\mathbf{r}, 3)$, for all i , if and only if the sequence $(T_{\pi(i)}(\mathbf{r}, 3))$, for $i = 2, 3, \dots, n+1$, consists of some traversal of this multigraph that touches every edge exactly once. By condition (2), the in-degree is equal to the out-degree at every vertex, and so the edges of the multigraph can be decomposed into disjoint cycles. Condition (3) states that the underlying graph is connected, and so any decomposition into disjoint cycles can be merged to give a single cycle, i.e. a traversal of the entire graph. $\square$

We can therefore construct σ as the union of various k -cycles, and only need to construct, given some i with $\sigma^k(i) = i$ and $i, \sigma(i), \dots, \sigma^{k-1}(i)$ distinct, the values $T_j^\top(\mathbf{e}_\ell)$, for $j = i, \sigma(i), \dots, \sigma^{k-1}(k)$.

To illustrate this, suppose we wish to construct a pair (σ', T'_i) that includes a given 5-cycle in the cycle decomposition of σ' . From our warm-up construction with $n = 6$, we have a pair σ, T_i defined by $\sigma(i) = i + 3$ and $T_i = \tau^{i-2}$, where σ is a 5-cycle, and this pair satisfies all four conditions on 5-friendly numbers. Let i be an index such that $(i, \sigma'(i), \dots, \sigma'^4(i))$ is the desired 5-cycle in the cycle decomposition of σ' . We then define $T'_{\sigma'^j(i)}(\mathbf{e}_\ell) = T_{\sigma^j(i)}^\top(\mathbf{e}_\ell)$, for $j = 0, 1, 2, 3, 4$. Now, from $\sigma(i) = i + 3$, we have $\sigma^j(i) = i + 3j$, taking the indices mod 5, and so we have

$$T'_{\sigma'^j(i)}(\mathbf{e}_\ell) = T_{\sigma^j(i)}^\top(\mathbf{e}_\ell) = T_{i+3j}^\top(\mathbf{e}_\ell) = \tau^{3j}(\mathbf{e}_\ell).$$

This defines the action of our matrix T'_i on the set $(i, \sigma'(i), \dots, \sigma'^4(i))$. We now show how to construct another pair (σ, T_i) corresponding to a cycle of order 31. Let

$$T_* := T_{\tau^3} + e_{3,5} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{pmatrix}$$

As it turns out, T_* has order 31 in $M_{5 \times 5}(\mathbb{F}_2)$, each nonzero vector in $\mathbb{F}_2^5$ occurs exactly once in the first row and once in the third row of the powers $T_*, T_*^2, \dots, T_*^{31}$, and the induced multigraph (in the sense of condition (3) above) is a single loop. We can therefore apply Lemma 4.1 to construct a σ and T_i for any σ that can be written as a disjoint union of 5-cycles and 31-cycles. Thus all integers which can be represented as $1 + 5\ell + 31m$, for ℓ and m non-negative, are 5-friendly. By the Chicken McNugget Theorem¹ it holds for all $n \geq 120$.

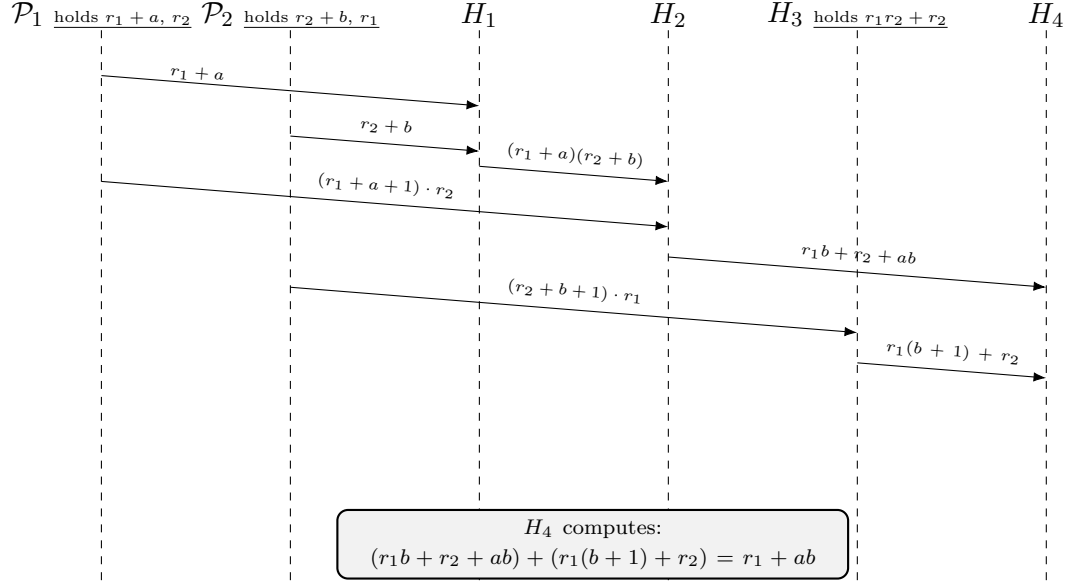
5 Information-theoretic cryptography with helper parties

In this section, we prove Theorem 1.3 and give our explicit construction of a 1-private multiparty computation protocol that can realize an arbitrary boolean functionality f computing a circuit C , which requires at most $7|C|$ helper parties.

To show that the randomness complexity of 1-private computation with helpers is > 1 , we can verify that the lower bound argument in [8] goes through even in the presence of helper parties. One way to see this is to note that, if there is only a single bit of randomness, all parties before the computation of $a_1 \cdots a_n$ can receive at most one message. In particular, any helper party can do nothing more than receive a single message, possibly XOR'ing the message with 1, and forwarding the message to another party since they have no private inputs. This behavior can be replicated by whichever party sent the message, and so the helper party can be removed to give a new protocol with one fewer helper party.

Protocol: Masked Multiplication with four Helpers $\Pi(\mathcal{P}_1, \mathcal{P}_2, a, b, r_1, r_2)$

¹Also known as **Frobenius number** for two coprime integers.



Masked Multiplication with four Helpers $\Pi(\mathcal{P}_1, \mathcal{P}_2, a, b, r_1, r_2)$

Input. Two parties $\mathcal{P}_1, \mathcal{P}_2$ hold the following data: $\mathcal{P}_1$ holds $r_1 + a$ and r_2 and $\mathcal{P}_2$ holds $r_2 + b$ and r_1 .
Helper parties $H_1, \dots, H_4$, where H_3 holds $r_1 r_2 + r_2$.

Output. H_4 holds $r_1 + ab$.

Protocol.

1. $\mathcal{P}_1$ sends $r_1 + a$ to H_1 .
2. $\mathcal{P}_2$ sends $r_2 + b$ to H_1 .
3. H_1 multiplies and sends $(r_1 + a) \cdot (r_2 + b)$ to H_2 .
4. $\mathcal{P}_1$ computes and sends $(r_1 + a + 1) \cdot r_2$ to H_2 .
5. H_2 adds to obtain $r_1 \cdot b + r_2 + a \cdot b$ and sends this result to H_4 .
6. $\mathcal{P}_2$ computes and sends $(r_2 + b + 1) \cdot r_1$ to H_3 .
7. H_3 adds their initial randomness to the message from $\mathcal{P}_2$ to obtain $r_1 \cdot (b + 1) + r_2$, and sends this result to H_4 .
8. H_4 adds the messages from H_2 and H_3 to obtain $r_1 + a \cdot b$, as desired.

Figure 4: Secure delegated multiplication with helper parties

Remark 1. The protocol in Figure 4 can be modified to require only 3 helper parties if we allow $\mathcal{P}_2$ to hold b, r_1, r_2 . Then the party H_3 could be removed, since $\mathcal{P}_2$ could compute H_3 's message $r_1 \cdot (b + 1) + r_2$ and send it to H_4 .

This is unhelpful in constructing a protocol for general circuits, since for intermediate wire values we will have the role of $\mathcal{P}_2$ played by a helper party, who cannot learn any wire value in the clear. If we wished to perform a more fine-grained accounting, this optimization would allow us to remove one helper party for each AND gate that has at least one input wire among its inputs.

Our proof of Theorem 1.3 relies on the following:

Lemma 5.1. *The protocol in Figure 4 is 1-private.*

Proof. We sketch the simulator argument for each party. $\mathcal{P}_1$ and $\mathcal{P}_2$ receive nothing other than random bits from the dealer, which can be perfectly simulated. H_1 receives two probabilistically independent values $r_1 + a$ and $r_2 + b$; these are indistinguishable from random bits s, t . H_4 likewise holds two independent random coin flips. We can in general test two values that are linear in r_1 and r_2 for independence by computing the determinant of their r_1 and r_2 coefficients. In the case of H_4 , we have

$$\det \begin{vmatrix} b & 1 \\ (b+1) & 1 \end{vmatrix} = 1$$

over $\mathbb{F}_2$.

H_2 can simulate their view by flipping two random coins u, v and outputting the pair of messages $(uv, (u+1)v)$. The values $uv, (u+1)v$ are each weighted coins which are 1 with probability $\frac{1}{4}$, and cannot both be equal to 1, since their product is zero. This fully defines their joint distribution, and the same holds for H_2 's messages, since

$$(r_1 + a) \cdot (r_2 + b) \cdot (r_1 + a + 1) \cdot r_2 = 0.$$

The helper party H_3 's messages can be simulated in the same way, since

$$(r_1 + 1) \cdot r_2 \cdot r_1 \cdot (r_2 + b + 1) = 0.$$

□

Proof of Theorem 1.3: For an arbitrary circuit C and active participants $\mathcal{P}_1, \dots, \mathcal{P}_n$, we assign $\mathcal{P}_1$ to be the dealer, and begin by instructing $\mathcal{P}_1$ to send (r_1, r_2) to every party $\mathcal{P}_i$. We then evaluate the circuit gate by gate, preserving the invariant that some party holds $r_1 + w$ for every wire value w . Initially the invariant is satisfied since $\mathcal{P}_i$ holds $r_1 + w_i$ for their input wire w_i .

To compute an AND gate where H_k holds $r_1 + w_k$ and H_ℓ holds $r_1 + w_\ell$, H_k sends $r_1 + w_k$ to a fresh helper party H'_k that holds r_2 . H_ℓ sends $r_1 + w_\ell$ to a fresh helper H'_ℓ holding r_2 , who in turn forwards $r_1 + r_2 + w_\ell$ to another party H''_ℓ holding r_1 . Then H'_k and H''_ℓ can play the roles of $\mathcal{P}_1$ and $\mathcal{P}_2$ in performing masked multiplication with four additional helpers, and the output party will hold $r_1 + w_k w_\ell$, preserving the invariant. This requires 7 helper parties per AND gate.

An XOR gate can be computed using three helper parties by replacing $r_1 + w_k$ and $r_1 + w_\ell$ with $(r_1 + r_2) + w_k$ and $r_2 + w_\ell$, and then adding.

Remark 2. It is worth exploring here what happens when we deploy the techniques from our proof of Theorem 1.1 on our masked multiplication with four helper parties. We could, as in our main construction, remove the helper parties by assigning other parties to play the role of a helper party for that multiplication. Since our masked multiplication protocol in Figure 4 requires only 2 bits of randomness, we might hope that this would give us another protocol with 5 or even fewer bits.

By Remark 1, we can use the modification of masked multiplication with three helpers, since $\mathcal{P}_2$ can hold their input bit b in the clear. But we still have three helper parties instead of only one, and so we will need a family of three permutations $\sigma_1, \sigma_2, \sigma_4$ with $\sigma_i : \{2, \dots, n\} \rightarrow \{2, \dots, n\}$ specifying which party plays the role of H_i in each masked multiplication. Each party will therefore help in a masked multiplication five times: as $\mathcal{P}_1, \mathcal{P}_2, H_1, H_2, H_4$. In particular, each party will hold two linear terms as $\mathcal{P}_2$, two as H_1 , and one each as H_2 and H_4 (the messages sent by H_2 and H_4 will certainly be learned by the party playing that role). These six linear terms must be independent, and so adapting the techniques from this paper to the masked multiplication protocol with three helper parties will require at least six bits of randomness.

Acknowledgments

Supported in part by NSF grants CNS-2246355, CCF-2220450, US-Israel BSF grant 2022370, and by Sunday Group.

References

- [1] Anilkumar, H.K.P., Narayanan, V., Prabhakaran, M., Prabhakaran, V.M.: Randomness in private sequential stateless protocols. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. *Lecture Notes in Computer Science*, vol. 15490, pp. 36–66. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_2
- [2] Canetti, R., Kushilevitz, E., Ostrovsky, R., Rosén, A.: Randomness vs. fault-tolerance. In: Burns, J.E., Attiya, H. (eds.) *Proceedings of the Sixteenth Annual ACM Symposium on Principles of Distributed Computing*, Santa Barbara, California, USA, August 21-24, 1997. pp. 35–44. ACM (1997). <https://doi.org/10.1145/259380.259416>
- [3] Couteau, G., Rosén, A.: Random sources in private computation. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security*, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 13791, pp. 443–473. Springer (2022). https://doi.org/10.1007/978-3-031-22963-3_15
- [4] Goyal, V., Ishai, Y., Song, Y.: Tight bounds on the randomness complexity of secure multiparty computation. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference*, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 13510, pp. 483–513. Springer (2022). https://doi.org/10.1007/978-3-031-15985-5_17

- [5] Ishai, Y., Kushilevitz, E., Li, X., Ostrovsky, R., Prabhakaran, M., Sahai, A., Zuckerman, D.: Robust pseudorandom generators. In: Fomin, F.V., Freivalds, R., Kwiatkowska, M.Z., Peleg, D. (eds.) Automata, Languages, and Programming - 40th International Colloquium, ICALP 2013, Riga, Latvia, July 8-12, 2013, Proceedings, Part I. Lecture Notes in Computer Science, vol. 7965, pp. 576–588. Springer (2013). https://doi.org/10.1007/978-3-642-39206-1_49
- [6] Ishai, Y., Sahai, A., Wagner, D.A.: Private circuits: Securing hardware against probing attacks. In: Boneh, D. (ed.) Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27
- [7] Kushilevitz, E., Mansour, Y.: Randomness in private computations. *SIAM J. Discret. Math.* **10**(4), 647–661 (1997). <https://doi.org/10.1137/S0895480196306130>
- [8] Kushilevitz, E., Ostrovsky, R., Prouff, E., Rosén, A., Thillard, A., Vergnaud, D.: Lower and upper bounds on the randomness complexity of private computations of AND. In: Hofheinz, D., Rosen, A. (eds.) Theory of Cryptography - 17th International Conference, TCC 2019, Nuremberg, Germany, December 1-5, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11892, pp. 386–406. Springer (2019). https://doi.org/10.1007/978-3-030-36033-7_15
- [9] Kushilevitz, E., Ostrovsky, R., Rosén, A.: Characterizing linear size circuits in terms of privacy. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996. pp. 541–550. ACM (1996). <https://doi.org/10.1145/237814.238002>